

CHAPTER 7 THE MODEL VIEW CONTROLLER FRAMEWORK, SYSTEM ORGANIZATION, AND DELIVERABLE II PREPARATION

7.1 WHY WEEK 7 MATTERS

Week 7 is an important turning point because the project is no longer only an idea, a diagram, or a simple prototype. By this stage, students already have:

-  a project purpose
-  requirements
-  user stories
-  UML models
-  interface screens
-  first CRUD logic
-  a first deliverable completed

That progress is useful, but it also creates a new problem. As the project grows, code can become messy. Forms begin to contain too much logic. Data rules can become repeated in several places. Interface code, data code, and business rules may start mixing together in a confusing way.

This is exactly why the Model-View-Controller framework becomes useful. MVC gives students a cleaner way to organize the system.

7.2 WHAT IS MODEL-VIEW-CONTROLLER?

Model-View-Controller, or MVC, is a software design framework that separates an application into three main parts:

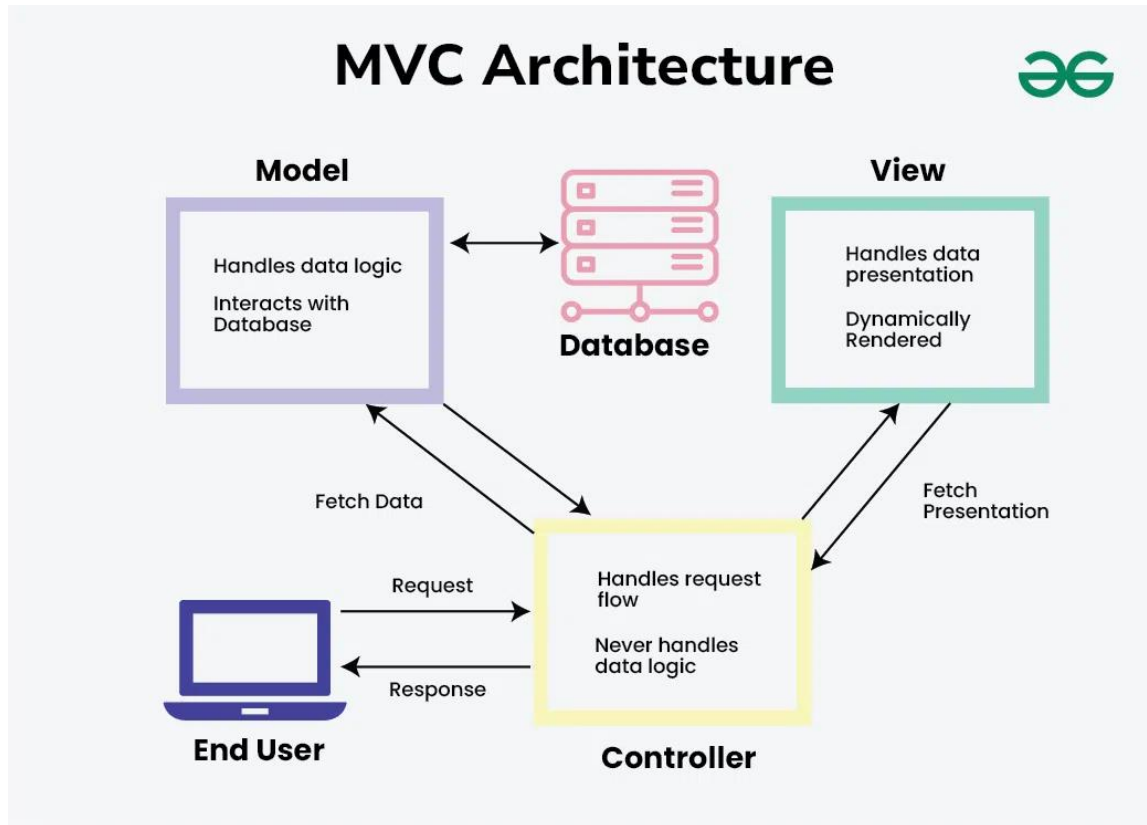
- the Model
- the View
- the Controller

Each part has a different responsibility.

The main idea is simple: instead of putting everything everywhere, MVC separates the system so that data, interface, and control logic do not become tangled together.

This makes the project easier to understand, easier to test, easier to improve, and easier to extend.

Figure 7.1: Classic MVC Architecture Diagram



[geeksforgeeks.org](https://www.geeksforgeeks.org)

MVC Architecture - System Design - GeeksforGeeks

7.3 WHY MVC IS IMPORTANT IN THIS COURSE

The course outline says Week 7 covers the MVC framework and also includes Deliverable II and a Project Presentation. That means Week 7 is not just about memorizing three definitions. It is about helping students build the second major stage of their project in a more structured and professional way.

In earlier weeks, students could survive with a simpler prototype style. A form with a few controls and some CRUD logic is acceptable for a first milestone. But by the second major deliverable, the system should begin showing stronger internal organization. If the project remains a collection of forms with mixed logic everywhere, it becomes harder to maintain and harder to evaluate as serious system development work.

MVC gives students a framework for structuring what they already built.

7.4 THE MODEL

The Model represents the data and the logic connected to that data.

In practical student terms, the Model usually includes:

-  classes that represent important entities
-  data values and properties
-  rules about valid data
-  operations that relate to those data objects

For example, in an advising appointment system, possible model classes might include:

- Student
- Advisor
- Appointment

The Appointment model might store:

- appointment identifier
- student number
- appointment date
- reason
- status

But the Model is more than a data container. It should also reflect the meaning of the data. For example, an appointment status should not become just random text with no rules. The Model should help represent the real structure and logic of the information.

This means the work from Week 6 Database Design connects directly into the Model. Primary keys, foreign keys, relationships, and CRUD-related data structure all support the Model part of MVC.

7.5 THE VIEW

The View is the part of the application the user sees.

This includes:

- forms
- labels
- text boxes
- buttons
- menus
- lists
- grids
- messages
- other interface elements

In this course, many students are building screens using Windows Forms in C#. In that context, the View is the visible form itself.

For example:

- the Appointment Request Form is a View
- the Advisor Dashboard is a View
- a login screen is a View
- a record list shown in a DataGridView is part of the View

The View should focus on presenting information and receiving user interaction. It should not become the place where all application logic is buried.

That is one of the biggest lessons of Week 7.

7.6 THE CONTROLLER

The Controller is the part that handles interaction between the View and the Model.

When the user clicks a button, submits a form, chooses an item, or performs another action, the Controller is responsible for deciding what should happen next.

For example:

- ✚ the user clicks Submit
- ✚ the Controller reads the input from the View
- ✚ the Controller checks or triggers rules related to the Model
- ✚ the Controller updates data
- ✚ the Controller then tells the View what should be shown next

In a beginner project, students may not always create a separate formal Controller class immediately, especially in simple Windows Forms work. But even then, the MVC mindset still matters. Students should begin separating responsibilities:

- ✚ the View displays
- ✚ the Model stores and represents data
- ✚ the Controller coordinates actions

This is much better than mixing all responsibilities in one large button-click method.

7.7 A SIMPLE WAY TO UNDERSTAND MVC

A practical way to understand MVC is this: The Model is the system's data reality. The View is the system's visible face. The Controller is the traffic manager between them.

If a student enters data into a form:

- ✚ the View shows the form
- ✚ the Controller handles the click or event
- ✚ the Model stores or updates the data

This separation makes the whole application more organized.

7.8 MVC IS ABOUT SEPARATION OF CONCERNS

One of the most important ideas behind MVC is separation of concerns.

Separation of concerns means different parts of the system should handle different jobs. When everything is mixed together, the system becomes harder to understand and harder to fix.

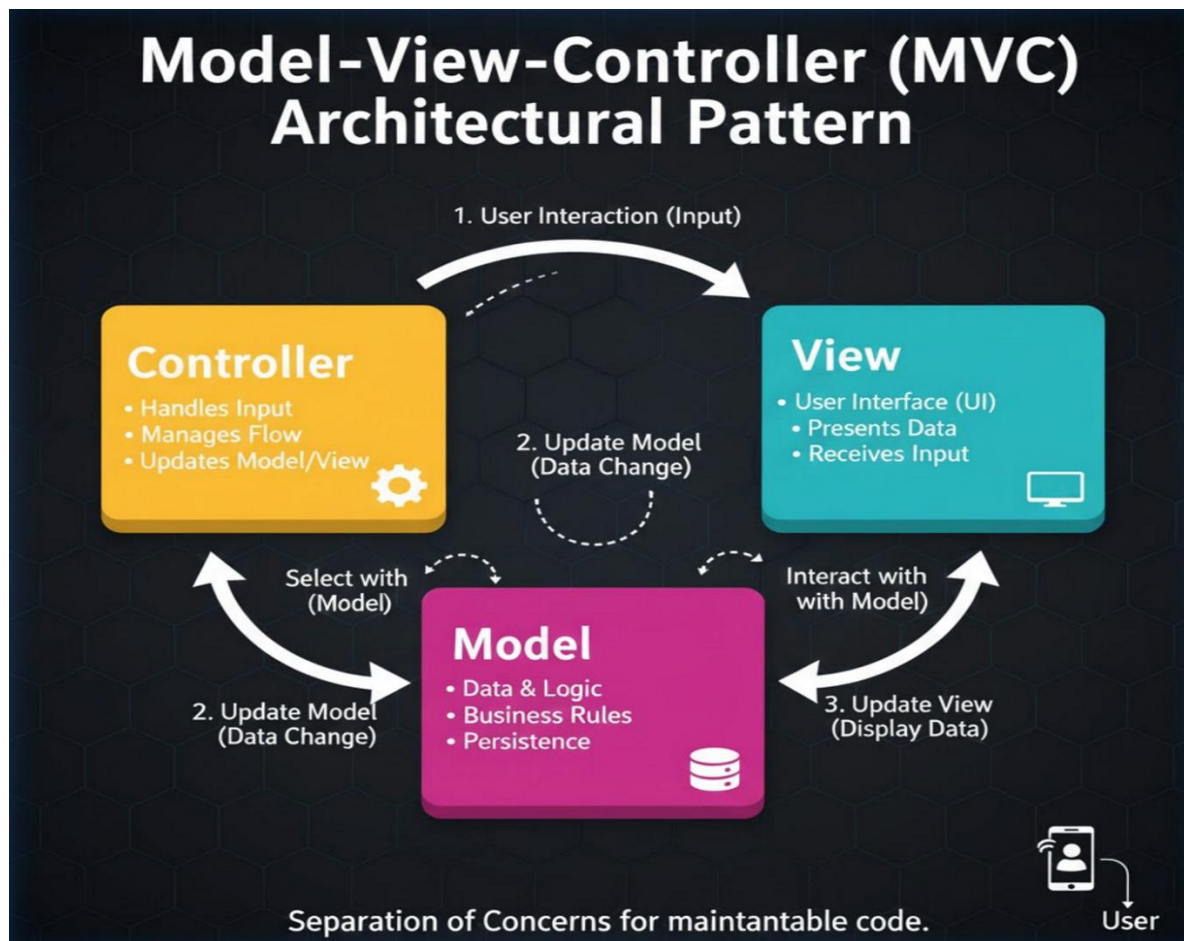
For example, imagine a single form that:

- ✚ stores all data directly
- ✚ validates all rules
- ✚ handles all navigation
- ✚ queries all records
- ✚ updates all status changes
- ✚ controls every screen transition

That may work at a tiny scale, but as the project grows, it becomes confusing. A change in one place can break something somewhere else.

MVC reduces that confusion by assigning clearer responsibilities.

Figure 7.2: MVC Separation of Concerns



python.plainenglish.io

Techniques for Keeping Layers Separate in Software Architecture | by Ramsha Suhail | Python in Plain English

Week 7 is an important turning point because the project is no longer only an idea, a diagram, or a simple prototype. By this stage, students already have:

- ✚ a project purpose
- ✚ requirements
- ✚ user stories
- ✚ Unified Modeling Language (UML) models
- ✚ interface screens
- ✚ first Create, Read, Update, and Delete (CRUD) logic
- ✚ a first deliverable completed

That progress is useful, but it also creates a new problem. As the project grows, code can become messy. Forms begin to contain too much logic. Data rules can become repeated in several places. Interface code, data code, and business rules may start mixing together in a confusing way.

This is exactly why the Model-View-Controller framework becomes useful. MVC gives students a cleaner way to organize the system.

7.2 What is Model-View-Controller?

Model-View-Controller, or MVC, is a software design framework that separates an application into three main parts:

- the **Model**
- the **View**
- the **Controller**

Each part has a different responsibility.

The main idea is simple: instead of putting everything everywhere, MVC separates the system so that data, interface, and control logic do not become tangled together. This makes the project easier to understand, easier to test, easier to improve, and easier to extend.

7.2.1 Detailed Analysis of the MVC Architectural Pattern Diagram (Figure 7.1)

The attached picture explains the Model-View-Controller architectural pattern. MVC is a software design structure that separates an application into three main parts: the Model, the View, and the Controller. The main idea of MVC is to avoid mixing data, interface code, and control logic in the same place. This makes the system easier to understand, easier to test, easier to improve, and easier to maintain.

In the System Development course, this topic fits very well in Week 7 because students already have requirements, user stories, UML diagrams, interface screens, basic CRUD logic, and a first project deliverable. MVC gives the project a cleaner internal structure before students move toward Deliverable II.

The Main Concept of the Diagram (Figure 7.1)

The picture shows that the user does not directly work with the database or the internal logic. The user interacts with the View, the View communicates user actions to the Controller, the Controller works with the Model, and then the result is shown back through the View.

In simple terms:

-  The **View** is what the user sees.
-  The **Controller** handles what the user does.
-  The **Model** contains the data, rules, and business logic.

This separation is called **separation of concerns**. Each part of the application has its own responsibility. The View should not do the job of the Model, the Model should not control the screen, and the Controller should not become a database table.

The View in the Diagram (Figure 7.1)

The View is the User Interface (UI). In C# Windows Forms, where C# is a Microsoft programming language used in the .NET environment, the View includes forms, buttons, text boxes, labels, menus, DataGridView tables, messages, and input fields.

In Windows Forms, the form file may still contain event handlers such as `btnSubmit_Click`. However, the important point is that this event handler should not contain all the business logic and database logic. Its job should mainly be to collect input from the form, pass it to the Controller, and display the result.

The Controller in the Diagram (Figure 7.1)

The Controller handles input, manages flow, and decides what should happen after a user action (e.g., button click). In beginner projects the logic may start inside `btnSubmit_Click`, but it is better to move it into a dedicated controller class.

The Model in the Diagram (Figure 7.1)

The Model represents the application's data and business rules. In an advising appointment system, the Model may include classes such as `Student`, `Advisor`, and `Appointment`. These classes describe the structure of the information used by the system. The Model is connected to persistence because the data usually needs to be saved in a database, but the Model itself should not be confused with the database. In a cleaner project structure, database operations are usually handled by a repository or data access layer that works with the Model. This helps students avoid placing SQL code directly inside Windows Forms.

Technical Example Using a Student Appointment System

Imagine a student wants to create a new advising appointment.

The **View** is the Windows Form: `AppointmentRequestForm` (contains `txtStudentNumber`, `txtReason`, `dateTimePickerAppointmentDate`, `btnSubmit`).

The Model:

C#

```
// Model class that represents the data structure of an Appointment entity in the MVC pattern
// Business rules related to appointments can be enforced here, in the Controller, or in a separate service class
public class Appointment
{
```

```

// Unique identifier for each appointment record (typically the primary key in the database)
public int AppointmentId { get; set; }

// Student identifier (foreign key linking to the Student table/entity)
public string StudentNumber { get; set; }

// Date and time when the appointment is scheduled
public DateTime AppointmentDate { get; set; }

// Description of why the student is requesting the appointment
public string Reason { get; set; }

// Current status of the appointment (e.g., "Pending", "Approved", "Cancelled", "Completed")
public string Status { get; set; }
}

```

The Controller:

C#

```

// Controller class responsible for coordinating actions between the View and the Model
public class AppointmentController
{
    // Processes a new appointment creation request
    // Receives an Appointment object populated from the View and returns a result message
    public string CreateAppointment(Appointment appointment)
    {
        // Check whether the Appointment object was created properly
        if (appointment == null)
        {
            return "Appointment information is missing.";
        }

        // Validation: Check if StudentNumber is null, empty, or contains only whitespace
        if (string.IsNullOrEmpty(appointment.StudentNumber))
        {
            // Return error message so the View can display it to the user
            return "Student number is required.";
        }
    }
}

```



```

// Remove unnecessary spaces before saving or processing the student number
appointment.StudentNumber = appointment.StudentNumber.Trim();

// Validation: Ensure the appointment date is today or in the future (business rule)
if (appointment.AppointmentDate.Date < DateTime.Today)
{
    // Return error message for invalid date
    return "Appointment date cannot be in the past.";
}

// Set default status for a newly created appointment
appointment.Status = "Pending";

// TODO, which means "to do" and is used as a developer reminder in code comments:
// In a full implementation, the Controller would call a repository
// or data access class here to save the Appointment object to the database
// (persistence logic)
// Example: appointmentRepository.AddAppointment(appointment);

// Return success message back to the View
return "Appointment created successfully.";
}
}

```

Example of how the View might call the Controller:

```

C#
// Event handler in the View (Windows Form) triggered when the user clicks the Submit button
private void btnSubmit_Click(object sender, EventArgs e)
{
    // Create a new Appointment object and populate it with data from the form controls
    Appointment newAppointment = new Appointment
    {
        StudentNumber = txtStudentNumber.Text, // Read value from the student number TextBox
        AppointmentDate = dtpAppointmentDate.Value, // Read value from the DateTimePicker control
        Reason = txtReason.Text // Read value from the reason TextBox
    }
}

```

```

};

// Instantiate the Controller
AppointmentController controller = new AppointmentController();

// Pass the populated Model object to the Controller method
string result = controller.CreateAppointment(newAppointment);

// Display the result message (success or error) back to the user via the View
MessageBox.Show(result);
}

```

The Flow Shown in the Diagram (Figure 7.1)

The picture shows a circular flow:

1. User Interaction (Input) → View
2. Controller processes the action and updates the Model
3. Model represents the data and business rules, while persistence is usually completed through a repository, data access class, or database layer.
4. Controller tells the View to refresh
5. View displays the updated data to the user

For beginner C# Windows Forms students, the safest and clearest way to understand the MVC flow is: **User → View → Controller → Model → Controller → View**

WHY THIS DIAGRAM IS IMPORTANT FOR WEEK 7

By Week 7, a student project should not only “work.” It should start becoming organized. A project that works but has all code inside one form is weak from a system development point of view. A project that separates the View, Model, and Controller (as shown in the commented examples above) is easier to explain, easier to improve, and more professional.

IMPORTANT TECHNICAL MEANING OF “SEPARATION OF CONCERNS”

“Separation of Concerns for maintainable code.” This is the most important message of the diagram. Without MVC, all logic lives inside the form. With MVC, responsibilities are clearly divided: View displays and collects input, Controller manages actions and flow, Model represents data and business rules.

many professional MVC implementations the Controller fully mediates updates. The safest understanding for C# Windows Forms beginners is: User → View → Controller → Model → Controller → View.

CONCEPTUAL SUMMARY OF THE DIAGRAM (FIGURE 7.1)

The attached picture shows MVC as a clean architectural pattern for organizing a growing software system. The commented code examples above demonstrate exactly how each layer works in practice. For this course, MVC is especially important because students are moving from early prototypes toward a more serious Deliverable II. Their projects should show better

structure, better code organization, and clearer responsibility between interface, data, and application logic. This is exactly why MVC is connected to Week 7 in the attached course material.

7.9 CONNECTING MVC TO EARLIER WEEKS

Week 7 does not replace the earlier course work. It builds on it.

From Week 2, students already know requirements, scope, constraints, and stakeholder needs. From Week 3, they already know user stories and iteration planning. From Week 4, they already know UML, especially class diagrams and behavior-oriented thinking. From Week 5, they already have interface screens and prototype work. From Week 6, they already have database design and CRUD foundations.

Now MVC helps organize all of that into a more professional structure.

This is why Week 7 appears exactly here in the semester. It is the right time to move from “my project works in some way” to “my project is becoming organized in a cleaner architectural way.”

7.10 MVC IN A STUDENT PROJECT EXAMPLE

Let us continue using the advising appointment system.

Model examples

- [Student class](#)
- [Advisor class](#)
- [Appointment class](#)

These classes represent the data and its structure.

View examples

-  Appointment Request Form
-  Advisor Dashboard
-  Appointment List Screen

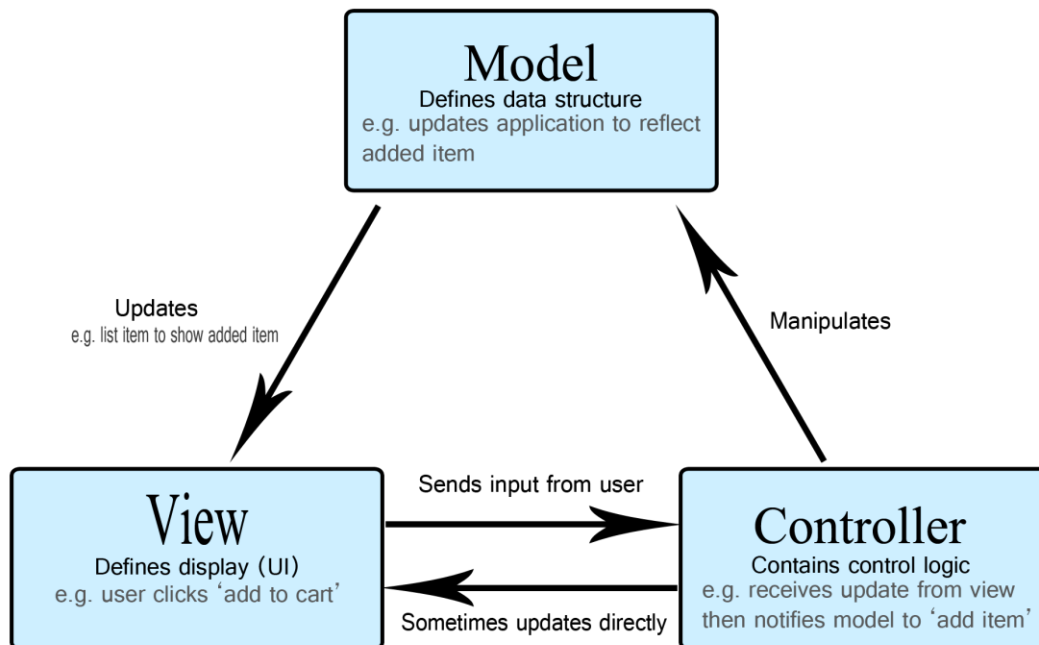
These are the visible screens users interact with.

Controller examples

-  logic that handles the Submit button
-  logic that loads appointments into the grid
-  logic that updates appointment status
-  logic that moves from one form to another

Even if the project is still simple, students should begin seeing these roles separately.

Figure 7.3: MVC Applied to an Advising Appointment System



developer.mozilla.org

MVC - Glossary | MDN

7.11 MVC DOES NOT MEAN ONE TECHNOLOGY ONLY

Students should be careful here. MVC is a framework or architectural pattern, not just one specific software tool.

Sometimes students hear MVC and think only about web frameworks. But the main educational goal here is broader. Week 7 is about learning the architectural idea of MVC, not only memorizing one platform-specific version.

That means students can still apply MVC thinking in projects built with Windows Forms and C# by separating:

- ✚ interface code
- ✚ data classes
- ✚ coordination logic

The exact technical implementation may vary, but the principle remains the same.

7.12 WHY MVC IMPROVES MAINTAINABILITY

A system is maintainable when it can be changed, fixed, improved, or extended without too much confusion.

MVC helps maintainability because changes in one area do not automatically destroy another area.

For example:

- ✚ if the interface layout changes, the Model should not need to be rewritten completely
- ✚ if a data rule changes, the View should not have to carry all that logic directly
- ✚ if navigation improves, the data structure should not collapse

This is important in student projects because Deliverable II should show progress beyond the first prototype. A messy project may still function, but it becomes difficult to extend.

7.13 WHY MVC IMPROVES TEAMWORK AND COMMUNICATION

Even though many students may work individually, MVC still matters because it reflects real project teamwork.

In real development environments:

- ✚ some people focus more on interface work
- ✚ some focus more on data and business logic
- ✚ some focus more on application flow and architecture

A well-separated system makes communication easier because different people can understand which part does what.

This also connects to the course objective that students should become familiar with what systems analysts do and with the techniques used in software development projects. MVC is one of those important organizational techniques.

7.14 MVC AND EVENT-DRIVEN PROGRAMMING

The course outline says the applications in this course involve event-driven programming. MVC works very naturally with event-driven systems.

When an event happens:

- ✚ the user clicks a button
- ✚ the form selection changes
- ✚ the user submits input

the Controller can respond to that event, access the Model, and update the View.

This makes MVC especially relevant for C# application work in this course.

7.15 MVC AND DATABASE WORK

By Week 6, students already worked with database design and CRUD. MVC now helps place those data operations in a cleaner structure.

For example:

- ✚ the Model represents the appointment and related data
- ✚ the Controller decides when Create, Read, Update, and Delete actions happen
- ✚ the View shows the results to the user

This means CRUD logic becomes less random and more organized.

If the project later moves toward stronger database implementation, MVC becomes even more valuable.

7.16 DELIVERABLE II IN WEEK 7

The course outline says Deliverable II belongs to Week 7 and represents 30% of the project, applying the second round of all phases of SDLC agile method, along with a Deliverable II Report and Project Presentation.

This means Week 7 is a major milestone.

Deliverable II should show more maturity than Deliverable I. Deliverable I could still feel like a first working prototype. Deliverable II should show:

- ✚ clearer organization
- ✚ stronger implementation
- ✚ improved project structure
- ✚ evidence of second-iteration thinking
- ✚ a more serious connection between interface, data, and system behavior

That is why MVC fits this week so well.

7.17 WHAT SHOULD IMPROVE FROM DELIVERABLE I TO DELIVERABLE II?

Students should understand that Deliverable II is not just “more of the same.” It should show improvement in quality and structure.

Possible improvements include:

- cleaner screen organization
- better naming and structure in code
- clearer separation between interface and data logic
- stronger CRUD behavior
- better testing and refinement
- more professional project presentation

In other words, Deliverable II should not only add features. It should show a better-designed system.

7.18 MVC AND REFACTORING

Refactoring means improving the internal structure of code without changing the main external behavior of the system.

Week 7 is a good time for refactoring because students may already notice that some of their first prototype code has become messy.

Examples of useful refactoring:

-  moving repeated data logic out of a form
-  improving naming of methods and controls
-  separating responsibilities more clearly
-  reducing duplicated validation code
-  organizing project files more logically

MVC supports this improvement mindset.

7.19 COMMON MISTAKES STUDENTS MAKE IN WEEK 7

There are several common problems. One is thinking MVC is only vocabulary and not something practical. Another is continuing to put all logic directly into the form code. Another is treating the Model as only a random class with no real structure. Another is building Deliverable II by adding features without improving organization. Another is talking about MVC in the report without actually showing any meaningful separation in the project. A final mistake is forgetting that Week 7 also includes Project Presentation, so students must be able to explain the structure of their project, not only show that it runs.

7.20 A SIMPLE EXAMPLE OF MVC THINKING IN AN APPOINTMENT SYSTEM

Imagine the student types information into an Appointment Request Form.

View The form contains:

-  student number text box
-  reason text box
-  submit button

Controller The submit event:

-  reads the values
-  validates them
-  creates or updates the appointment data
-  refreshes the screen

Model The Appointment object stores:

-  appointment identifier
-  student number
-  reason
-  status

This may still be a simple prototype, but the student is already thinking in MVC terms.

7.21 PROJECT PRESENTATION IN WEEK 7

Because Week 7 includes Project Presentation, students should be prepared to explain:

- what their project does
- what improved since Deliverable I
- how the system is now more organized
- where the Model, View, and Controller responsibilities appear
- what still needs work in the next iteration

A good Week 7 presentation should show not only progress, but better structure.

7.22 DELIVERABLE II REPORT

The Deliverable II Report should not just repeat old material from Week 5. It should show the second round of development and explain how the project has become stronger.

A useful Deliverable II Report should include:

- project title and short reminder of purpose
- main users
- what changed since Deliverable I
- architecture or structure improvements
- interface improvements
- CRUD or data improvements
- how MVC thinking has been applied
- what remains for the next phase

This is where students show that the project is evolving in a disciplined way.

7.23 HOW WEEK 7 PREPARES FOR LATER WEEKS

Week 7 prepares students for later topics such as:

- ✚ stronger database implementation
- ✚ deployment strategy thinking
- ✚ output reports
- ✚ final deliverables and presentations

Without a cleaner structure, those later steps become harder. MVC gives students a bridge between prototype work and more mature implementation.

7.24 WHAT STUDENTS SHOULD KNOW BY THE END OF CHAPTER 7

By the end of this chapter, students should be able to explain what the Model-View-Controller framework is and why it matters. They should be able to distinguish the responsibilities of the Model, the View, and the Controller. They should understand how MVC helps organize user interface work, CRUD behavior, and project growth more cleanly. They should also understand how Week 7 supports Deliverable II, Project Presentation, and the Deliverable II Report.

7.25 CHAPTER SUMMARY

Chapter 7 introduces the Model-View-Controller framework as a cleaner way to organize a growing system. Students learn that a successful project is not only about adding features. It is also about separating responsibilities so the system becomes easier to maintain, explain, and improve.

This chapter connects directly to the second major project milestone in the course. Week 7 is where Deliverable II should show not only more work, but better structure. MVC helps students move from an early working prototype toward a more professional application design. That is exactly why this topic belongs at this stage of the semester.

REFERENCES

- Vanier College, Faculty of Continuing Education. (2025). *Course Outline: System Development (420-940-VA)*. Vanier College.
- Shelly, G. B., Cashman, T. J., & Rosenblatt, H. J. (2014). *System Analysis and Design* (10th ed.). Course Technology. ISBN: 9781285171340.
- Kendall, K. E., & Kendall, J. E. (2023). *Systems Analysis and Design*. Pearson Education. ISBN-13: 9780137947850.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley. ISBN: 9780321127426.
- Microsoft Learn. (2025–2026). *Model-View-Controller (MVC) in .NET*. Microsoft Learn.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. ISBN: 9780201633610.